Creating a Reusable Web Component

Web Components[169] are an industry standard way of extending HTML itself to implement custom behaviors and presentation.

You don't need to start from scratch when building a web component. You can build upon a rich ecosystem of npm[170] packages. We'll make use of lit.[171] We start by "pinning" it to our application so that it can be imported:

```
> bin/importmap pin lit
Pinning "lit" to https://.../index.js
Pinning "@lit/reactive-element" to https://.../reactive-element.js
Pinning "lit-element/lit-element.js" to https://.../lit-element.js
Pinning "lit-html" to https://.../lit-html.js
Pinning "lit-html/is-server.js" to https://.../is-server.js
```

Next, we'll write a web component. The following example renders the current time in blue. Create a directory named `app/javascript/elements` and create a file named `current-time.js` in that directory with the following contents:

```
import {html, css, LitElement} from 'lit';

class CurrentTime extends LitElement {
  static styles = css`span { color: blue }`;

  render() {
    return html`<span>${new Date().toLocaleTimeString()}</span>`;
  }
}

customElements.define('current-time', CurrentTime);
```

This code imports three properties from the `lit` package and defines a class that extends `LitElement` by defining a style that's scoped to this single element function that returns an HTML fragment. Finally, a new custom element is defined and associated with this class.